

未命名笔记

1. 为什么要做 MtA

ECDSA 签名步骤如下：

$k \leftarrow \mathbb{Z}_n^*$ 。 $R := kG$, $h := R.x \bmod n$ 。 $s := k^{-1}(m + h \cdot \mathbf{sk})$ 。 输出 (R, s) 。

这里 n 是曲线的阶，通常是质数；本文假设其为质数。

在 MPC 中，各参与方有随机的 k_i ，很容易算出 R 。然而算 s 的时候要算 k^{-1} 。如果直接把 k_i 加起来，那就泄露了 k ，从而泄露私钥。因此需要对 s 进行变形。

设非 0 随机数 γ 。则

$$s = (k\gamma)^{-1}(m\gamma + \mathbf{sk} \cdot h\gamma). \quad (1)$$

有了这个变形，MtA (Multiplication to Addition) 马上就要登场了。我们不需要预先指定 k, γ ，而是采取一种“先打枪再画靶子”的方案：

1.1. MtA 理想功能

我们要求各方生成随机秘密 k_i ，约定 $k = \sum_i k_i$ 。类似地，各方生成随机 γ_i ，约定 $\gamma = \sum_i \gamma_i$ 。

然后，对于有序的一对参与方 (i, j) ，（这里 i, j 遍历所有参与方），我们假设存在 MtA 协议，使得参与方 i 得到秘密 $a_{i,j}$ ，参与方 j 得到秘密 $b_{i,j}$ 。协议的理想功能可以简述为下式：

$$a_{i,j} + b_{i,j} := k_j \gamma_i. \quad (\text{mta})$$

公式 (mta) 中, 各变量的归属如下:

γ_i 是参与方 i 的秘密输入, $a_{i,j}$ 是参与方 i 的秘密输出。 k_j 是参与方 j 的秘密输入, $b_{i,j}$ 是参与方 j 的秘密输出。

当 $i = j$ 时无需跑协议。参与方 i 只需在本地令 $a_{i,i} := k_i \gamma_i$, $b_{i,i} := 0$ 。

1.2. 交换 a, b 份额

MtA 理想功能只是算出了 a, b 秘密份额。各参与方还需交换这些份额才能算出 $k\gamma$ 。交换方式如下。

首先, 对每个参与方 i , 他持有与其他 $j \neq i$ 生成的份额 $a_{i,j}$, $b_{j,i}$ (注意不是 $b_{i,j}$), 以及本地份额 $a_{i,i}$, $b_{i,i}$ 。他计算 σ_i 如下式。

$$\sigma_i = \sum_j (a_{i,j} + b_{j,i}). \quad (\text{ex1})$$

然后, 参与方 i 与所有参与方明文交换 σ_j , 求和得到 $k\gamma$ 如下式。各方如果诚实, 必然得到相同的 $k\gamma$ 。

$$k\gamma = \sum_j \sigma_j. \quad (\text{ex2})$$

提示: 之所以 σ_i 可以明文交换, 是因为加项分解是信息论安全的。这保证了具体的 a, b 份额不会泄露。

验算一下公式 (mta, ex1, ex2) 的正确性。

$$\sum_{i,j} (a_{i,j} + b_{i,j}) = \sum_i (a_{i,i} + b_{i,i}) + \sum_{j \neq i} (a_{i,j} + b_{i,j}) \quad (2)$$

$$= \sum_i k_i \gamma_i + \sum_{j \neq i} k_j \gamma_i \quad (3)$$

$$= \sum_{i,j} k_j \gamma_i \quad (4)$$

$$= k\gamma. \quad (5)$$

仿照公式 (mta, ex1, ex2), 也可以算出 $sk \cdot \gamma$ 。

今有 Sender, Receiver 两参与方, 分别持有秘密值 $x_a \bmod n, x_b \bmod n$ 。他们要得到另外的秘密值 y_a, y_b , 使得

$$y_a + y_b := x_a x_b \pmod{n}. \quad (6)$$

怎么做? 以下两章介绍一套朴素的办法如下, 称为“朴素的位分解 OT”。术语 OT = Oblivious Transfer, 翻译为“不经意传输”。至于朴素在哪, 本章末尾会与后续笔记对比说明。

2. 朴素位分解

2.1. 规格 ($\approx$ 理想功能)

输入: Sender 的 x_a , Receiver 的 x_b 。

输出: Sender 的 y_a , Receiver 的 y_b , 满足

$$y_a + y_b := x_a x_b \pmod{n}. \quad (7)$$

双方固定 x_a, x_b 的最大比特长度 ℓ 。如此, 比特下标 t 取整数 0 到 $\ell - 1$ 。若 x_a, x_b 比特数不足, 就在高位补 0。

安全承诺: 除了自己的输出份额, 双方对彼此的输入一无所知, 即

- Sender 得不到关于 x_b 的任何信息;
- Receiver 得不到关于 x_a 的任何信息。

份额本身不泄密： y_a, y_b 单独看都是均匀随机数，只有相加才能还原 $x_a x_b$ 。

安全假设：

- 双方半诚实，即都遵守协议规则。恶意方能造成什么破坏，见末章“朴素在哪”。
- 依赖子协议：每个比特跑一次二选一 OT（下一章），共 ℓ 次，假设其安全承诺成立。OT 藏住各比特 d_t ，就是藏住 x_b ；OT 藏住没选中的那条消息， x_a 就被现摇的 r_t 一次一密地掩盖着。

2.2. 实施

(0) Receiver 获取 x_b 的二进制表示，记为

$$\sum_{t=0}^{\ell-1} d_t 2^t = x_b. \quad (\text{deco-repr})$$

(1) Sender 对每个 t ，摇随机秘密 $r_t \leftarrow \mathbb{Z}_n$ 。生成如下两条消息。

$$m_{t,0} := r_t \quad (8)$$

$$m_{t,1} := r_t + x_a 2^t \bmod n. \quad (\text{deco-msg})$$

(2) 对每个 t ，双方执行一次二选一 OT 协议（下一节会讲）。之后，Receiver 得到 m_{t,d_t} 如下式。

$$m_{t,d_t} := r_t + d_t x_a 2^t \bmod n. \quad (\text{deco-md})$$

(3) Sender 和 Receiver 计算各自的本地份额

$$y_a := - \sum_t m_{t,0} \bmod n; \quad (9)$$

$$y_b := \sum_t m_{t,d_t} \bmod n. \quad (\text{deco-out})$$

位分解协议至此结束。验算一下。

$$y_b = \sum_t (r_t + d_t x_a 2^t) \quad (10)$$

$$= \sum_t r_t + x_a \sum_t d_t 2^t \quad (11)$$

$$= -y_a + x_a x_b. \quad (12)$$

3. 朴素二选一 OT 协议

3.1. 规格 ($\approx$ 理想功能)

输入：Sender 的两条消息 m_0, m_1 ；Receiver 的选择 $d \in \{0, 1\}$ 。

输出：Receiver 得到 m_d ；Sender 无输出。

安全承诺：Receiver 得不到另一条消息，Sender 不知道选什么。即：Receiver 得不到 m_{1-d} ，Sender 得不到 d 。

安全假设：

- 双方半诚实，即都遵守协议规则。
- “Receiver 得不到 m_{1-d} ” 是计算安全：主要依赖椭圆曲线上的 CDH 难题，详见 § 实施末尾小结。此外，Hash 视作随机预言机，对称加密 Enc 可靠。
- “Sender 得不到 d ” 是统计安全，见 § 实施 (2) 末尾的提示。

3.2. 实施

(1) Sender 摇随机数 $\alpha \leftarrow \mathbb{Z}_n^*$ 。计算下式，发给 Receiver。

$$A := \alpha G. \quad (\text{ot-acom})$$

(2) Receiver 摇随机数 $\beta \leftarrow \mathbb{Z}_n^*$ 。计算下式，发给 Sender。虽然公式里已经体现出来，但还是要强调，只发所选的那一个。

$$B := \begin{cases} \beta G & \text{if } d = 0, \\ \beta G + A & \text{if } d = 1. \end{cases} \quad (\text{ot-bcom})$$

提示： βG 几乎是均匀随机的。Sender 无法区分收到的是 βG 还是 $\beta G + A$ 。

(3) Sender 计算两个密钥

$$K_0 := \text{Hash}(\alpha B), \quad (13)$$

$$K_1 := \text{Hash}(\alpha(B - A)). \quad (\text{ot-keys})$$

Sender 计算如下两个对称加密的密文，发给 Receiver。

$$C_0 := \text{Enc}(K_0, m_0), \quad (14)$$

$$C_1 := \text{Enc}(K_1, m_1). \quad (\text{ot-cts})$$

(4) Receiver 计算一个密钥。

$$K_d := \text{Hash}(\beta A). \quad (15)$$

用这个密钥解开 C_d 得到 m_d ，解不开 C_{1-d} 。协议至此结束。

(小结) 实际上，在 $d = 0$ 时，

$$K_0 = \text{Hash}(\alpha\beta G), K_1 = \text{Hash}(\alpha\beta G - \alpha^2 G). \quad (16)$$

而在 $d = 1$ 时，

$$K_0 = \text{Hash}(\alpha\beta G + \alpha^2 G), K_1 = \text{Hash}(\alpha\beta G). \quad (17)$$

Receiver 无法计算 $\pm\alpha^2 G$ ，使得 Receiver 算不出另一个密钥。Sender 无法区分 βG 和 $\beta G + A$ ，使得 Sender 不知道对方选的是什么。

4. 朴素在哪

前文提出的“基于位分解和 2 选 1 OT 的 MtA”功能完备，正确性也没有问题。说它朴素，是和后续笔记（01 至 07）对比出来的。短板有两条。

(1) 太贵：椭圆曲线运算量跟着比特数走，且无法摊销。

每个比特 t 都要跑一次基于椭圆曲线的二选一 OT，一次 OT 约有 5 次点乘 (αG , βG , αB , $\alpha(B - A)$, βA)。一次 MtA 有 $\ell \approx 256$ 个比特；每次签名，每个有序参与方对 (i, j) 要跑 2 个 MtA ($k\gamma$ 和 $\mathbf{sk} \cdot \gamma$ 各一个)。也就是说，每签一次名，每对参与方就要做几千次点乘。而且 α, β 都是现摇的，没有任何东西能跨签名复用。

后续笔记的路线是“OT 扩展”：椭圆曲线只在 keygen 阶段跑 $\kappa = 256$ 次 base OT (Endemic OT)，生成可长期保存的种子 (PPRF 与 GGM 树)；签名阶段吃这些种子，用哈希/异或等对称运算扩展出任意多的 OT 实例 (IKNP03 OT 扩展, SoftSpokenOT)。对称运算比椭圆曲线便宜若干数量级，详见 IKNP03 OT 扩展的小结（密钥交换“基于异或运算的消去律”而非椭圆曲线），以及随机 VOLE 末尾的讨论“Keygen 真正摊销的是什么”。

(2) 只防半诚实，不防恶意：协议没有任何一致性检查。

本章协议处处信任对方遵守协议规则。一个典型攻击：恶意 Sender 在第 t^* 位用 $x'_a \neq x_a$ 构造 (deco-msg)，其余位诚实。则两份额之和变成

$$y_a + y_b = x_a x_b + d_{t^*} (x'_a - x_a) 2^{t^*}. \quad (18)$$

当 $d_{t^*} = 0$ 时下游签名照常成功，当 $d_{t^*} = 1$ 时签名失败。Sender 观察成败，就白得了 x_b 的一个比特，代价只是一场失败的会话。这叫 selective failure 攻击。多场会话反复试探的后果，详见 KOS15 恶意安全的“追问 2”。

后续笔记的一致性检查正是冲着这类攻击去的：随机 VOLE 的校验负载抓在不

同 OT 实例上使用不一致 w 的 Sender，与上述攻击同型；KOS15 恶意安全的挑战-响应则抓 OT 扩展里篡改 u 矩阵的 Receiver。顺带一提，就连 base OT 本身，DKLs23 也不用本章的朴素二选一 OT，而是用安全定义更清晰的 Endemic OT (Endemic OT)。